printed at home, october 2026

13 essays, 38,921 words. all of them are at paulgraham.com, free, as they should be. this is one reader's copy, printed at home to be read away from a screen, with a pencil.

a printed book of essays
github.com/workinprogmess/a-printed-book-of-essays

01

how to get startup ideas

paulgraham.com/startupideas.html

people want a large amount. choose the latter. not all ideas of that type are good startup ideas, but nearly all good startup ideas are of that type.

imagine a graph whose x axis represents all the people who might want what you're making and whose y axis represents how much they want it. if you invert the scale on the y axis, you can envision companies as holes. google is an immense crater: hundreds of millions of people use it, and they need it a lot. a startup just starting out can't expect to excavate that much volume. so you have two choices about the shape of hole you start with. you can either dig a hole that's broad but shallow, or one that's narrow and deep, like a well.

made-up startup ideas are usually of the first type. lots of people are mildly interested in a social network for pet owners.

nearly all good startup ideas are of the second type. microsoft was a well when they made altair basic. there were only a couple thousand altair owners, but without this software they were programming in machine language. thirty years later facebook had the same shape. their first site was exclusively for harvard students, of which there are only a few thousand, but those few thousand users wanted it a lot.

when you have an idea for a startup, ask yourself: who wants this right now? who wants this so much that they'll use it even when it's a crappy version one made by a two-person startup they've never heard of? if you can't answer that, the idea is probably bad. [3]

you don't need the narrowness of the well per se. it's depth you need; you get narrowness as a byproduct of optimizing for depth (and speed). but you almost always do get it. in practice the link between depth and narrowness is so strong that it's a good sign when you know that an idea will appeal strongly to a specific group or type of user.

but while demand shaped like a well is almost a necessary condition for a good startup idea, it's not a sufficient one. if mark zuckerberg had built something that could only ever have appealed to harvard students, it would not have been a good startup idea. facebook was a good idea because it started with a small market there was a fast path out of. colleges are similar enough that if you build a facebook that works at harvard, it will work at any college. so you spread rapidly through all the colleges. once you have all the college students, you get everyone else simply by letting them in.

similarly for microsoft: basic for the altair; basic for other machines; other languages besides basic; operating systems; applications; ipo.

self

how do you tell whether there's a path out of an idea? how do you tell whether something is the germ of a giant company, or just a niche product? often you can't. the founders of airbnb didn't realize at first how big a market they were tapping. initially they had a much narrower idea. they were going to let hosts rent out space on their floors during conventions. they didn't foresee the expansion of this idea; it forced itself upon them gradually. all they knew at first is that they were onto something. that's probably as much as bill gates or mark zuckerberg knew at first.

occasionally it's obvious from the beginning when there's a path out of the initial niche. and sometimes i can see a path that's not immediately obvious; that's one of our specialties at yc. but there are limits to how well this can be done, no matter how much experience you have. the most important thing to understand about paths out of the initial idea is the meta-fact that these are hard to see.

knowing how to hack also means that when you have ideas, you'll be able to implement them. that's not absolutely necessary (jeff bezos couldn't) but it's an advantage. it's a big advantage, when you're considering an idea like putting a college facebook online, if instead of merely thinking "that's an interesting idea," you can think instead "that's an interesting idea. i'll try building an initial version tonight." it's even better when you're both a programmer and the target user, because then the cycle of generating new versions and testing them on users can happen inside one head.

noticing

once you're living in the future in some respect, the way to notice startup ideas is to look for things that seem to be missing. if you're really at the leading edge of a rapidly changing field, there will be things that are obviously missing. what won't be obvious is that they're startup ideas. so if you want to find startup ideas, don't merely turn on the filter "what's missing?" also turn off every other filter, particularly "could this be a big company?" there's plenty of time to apply that test later. but if you're thinking about that initially, it may not only filter out lots of good ideas, but also cause you to focus on bad ones.

most things that are missing will take some time to see. you almost have to trick yourself into seeing the ideas around you.

but you *know* the ideas are out there. this is not one of those problems where there might not be an answer. it's impossibly unlikely that this is the exact moment when technological progress stops. you can be sure people are going to build things in the next few years that will make you think "what did i do before x?"

and when these problems get solved, they will probably seem flamingly obvious in retrospect. what you need to do is turn off the filters that usually prevent you from seeing them. the most powerful is simply taking the current state of the world for granted. even the most radically open-minded of us mostly do that. you couldn't get from your bed to the front door if you stopped to question everything.

but if you're looking for startup ideas you can sacrifice some of the efficiency of taking the status quo for granted and start to question things. why is your inbox overflowing? because you get a lot of email, or because it's hard to get email out of your inbox? why do you get so much email? what problems are people trying to solve by sending you email? are there better ways to solve them? and why is it hard to get emails out of your inbox? why do you keep emails around after you've read them? is an inbox the optimal tool for that?

pay particular attention to things that chafe you. the advantage of taking the status quo for granted is not just that it makes life (locally) more efficient, but also that it makes life more tolerable. if you knew about all the things we'll get in the next 50 years but don't have yet, you'd find present day life pretty constraining, just as someone from the present would if they were sent back 50 years in a time machine. when something annoys you, it could be because you're living in the future.

when you find the right sort of problem, you should probably be able to describe it as *obvious*, at least to you. when we started viaweb, all the online stores were built by hand, by web designers making individual html pages. it was obvious to us as programmers that these sites would have to be generated by software. [5]

which means, strangely enough, that coming up with startup ideas is a question of seeing the obvious. that suggests how weird this process is: you're trying to see things that are obvious, and yet that you hadn't seen.

so if you're a cs major and you want to start a startup, instead of taking a class on entrepreneurship you're better off taking a class on, say, genetics. or better still, go work for a biotech company. cs majors normally get summer jobs at computer hardware or software companies. but if you want to find start-up ideas, you might do better to get a summer job in some unrelated field. [8]

or don't take any extra classes, and just build things. it's no coincidence that microsoft and facebook both got started in january. at harvard that is (or was) reading period, when students have no classes to attend because they're supposed to be studying for finals. [9]

but don't feel like you have to build things that will become startups. that's premature optimization. just build things. preferably with other students. it's not just the classes that make a university such a good place to crank oneself into the future. you're also surrounded by other people trying to do the same thing. if you work together with them on projects, you'll end up producing not just organic ideas, but organic ideas with organic founding teams — and that, empirically, is the best combination.

beware of research. if an undergrad writes something all his friends start using, it's quite likely to represent a good startup idea. whereas a phd dissertation is extremely unlikely to. for some reason, the more a project has to count as research, the less likely it is to be something that could be turned into a startup. [10] i think the reason is that the subset of ideas that count as research is so narrow that it's unlikely that a project that satisfied that constraint would also satisfy the orthogonal constraint of solving users' problems. whereas when students (or professors) build something as a side-project, they automatically gravitate toward solving users' problems — perhaps even with an additional energy that comes from being freed from the constraints of research.

competition

because a good idea should seem obvious, when you have one you'll tend to feel that you're late. don't let that deter you. worrying that you're late is one of the signs of a good idea. ten minutes of searching the web will usually settle the question. even if you find someone else working on the same thing, you're probably not too late. it's exceptionally rare for startups to be killed by competitors — so rare that you can almost discount the possibility. so unless you discover a competitor with the sort of lock-in that would prevent users from choosing you, don't discard the idea.

if you're uncertain, ask users. the question of whether you're too late is subsumed by the question of whether anyone urgently needs what you plan to make. if you have something that no competitor does and that some subset of users urgently need, you have a beachhead. [11]

the question then is whether that beachhead is big enough. or more importantly, who's in it: if the beachhead consists of people doing something lots more people will be doing in the future, then it's probably big enough no matter how small it is. for example, if you're building something differentiated from competitors by the fact that it works on phones, but it only works on the newest phones, that's probably a big enough beachhead.

err on the side of doing things where you'll face competitors. inexperienced founders usually give competitors more credit than they deserve. whether you succeed depends far more on you than on your competitors. so better a good idea with competitors than a bad one without.

you don't need to worry about entering a "crowded market" so long as you have a thesis about what everyone else in it is overlooking. in fact that's a very promising starting point. google was that type of idea. your thesis has to be more precise than "we're going to make an x that doesn't suck" though. you have to be able to phrase it in terms of something the incumbents are overlooking. best of all is when you can say that they didn't have the courage of their convictions, and that your plan is what they'd

while the best way to discover startup ideas is to become the sort of person who has them and then build whatever interests you, sometimes you don't have that luxury. sometimes you need an idea now. for example, if you're working on a startup and your initial idea turns out to be bad.

for the rest of this essay i'll talk about tricks for coming up with startup ideas on demand. although empirically you're better off using the organic strategy, you could succeed this way. you just have to be more disciplined. when you use the organic method, you don't even notice an idea unless it's evidence that something is truly missing. but when you make a conscious effort to think of startup ideas, you have to replace this natural constraint with self-discipline. you'll see a lot more ideas, most of them bad, so you need to be able to filter them.

one of the biggest dangers of not using the organic method is the example of the organic method. organic ideas feel like inspirations. there are a lot of stories about successful startups that began when the founders had what seemed a crazy idea but "just knew" it was promising. when you feel that about an idea you've had while trying to come up with startup ideas, you're probably mistaken.

when searching for ideas, look in areas where you have some expertise. if you're a database expert, don't build a chat app for teenagers (unless you're also a teenager). maybe it's a good idea, but you can't trust your judgment about that, so ignore it. there have to be other ideas that involve databases, and whose quality you can judge. do you find it hard to come up with good ideas involving databases? that's because your expertise raises your standards. your ideas about chat apps are just as bad, but you're giving yourself a dunning-kruger pass in that domain.

the place to start looking for ideas is things you need. there *must* be things you need. [14]

one good trick is to ask yourself whether in your previous job you ever found yourself saying "why doesn't someone make x? if someone made x we'd buy it in a second." if you can think of any x people said that about, you probably have an idea. you know there's demand, and people don't say that about things that are impossible to build.

more generally, try asking yourself whether there's something unusual about you that makes your needs different from most other people's. you're probably not the only one. it's especially good if you're different in a way people will increasingly be.

if you're changing ideas, one unusual thing about you is the idea you'd previously been working on. did you discover any needs while working on it? several well-known startups began this way. hotmail began as something its founders wrote to talk about their previous startup idea while they were working at their day jobs. [15]

a particularly promising way to be unusual is to be young. some of the most valuable new ideas take root first among people in their teens and early twenties. and while young founders are at a disadvantage in some respects, they're the only ones who really understand their peers. it would have been very hard for someone who wasn't a college student to start facebook. so if you're a young founder (under 23 say), are there things you and your friends would like to do that current technology won't let you?

the next best thing to an unmet need of your own is an unmet need of someone else. try talking to everyone you can about the gaps they find in the world. what's missing? what would they like to do that they can't? what's tedious or annoying, particularly in their work? let the conversation get general; don't be trying too hard to find startup ideas. you're just looking for something to spark a thought. maybe you'll notice a problem they didn't consciously realize they had, because you know how to solve it.

but talking about looking explicitly for waves makes it clear that such recipes are plan b for getting startup ideas. looking for waves is essentially a way to simulate the organic method. if you're at the leading edge of some rapidly changing field, you don't have to look for waves; you are the wave.

finding startup ideas is a subtle business, and that's why most people who try fail so miserably. it doesn't work well simply to try to think of startup ideas. if you do that, you get bad ones that sound dangerously plausible. the best approach is more indirect: if you have the right sort of background, good startup ideas will seem obvious to you. but even then, not immediately. it takes time to come across situations where you notice something missing. and often these gaps won't seem to be ideas for companies, just things that would be interesting to build. which is why it's good to have the time and the inclination to build things just because they're interesting.

live in the future and build what seems interesting. strange as it sounds, that's the real recipe.

---

notes

1 this form of bad idea has been around as long as the web. it was common in the 1990s, except then people who had it used to say they were going to create a portal for x instead of a social network for x. structurally the idea is stone soup: you post a sign saying "this is the place for people interested in x," and all those people show up and you make money from them. what lures founders into this sort of idea are statistics about the millions of people who might be interested in each type of x. what they forget is that any given person might have 20 affinities by this standard, and no one is going to visit 20 different communities regularly.

2 i'm not saying, incidentally, that i know for sure a social network for pet owners is a bad idea. i know it's a bad idea the way i know randomly generated dna would not produce a viable organism. the set of plausible sounding startup ideas is many times larger than the set of good ones, and many of the good ones don't even sound that plausible. so if all you know about a startup idea is that it sounds plausible, you have to assume it's bad.

3 more precisely, the users' need has to give them sufficient activation energy to start using whatever you make, which can vary a lot. for example, the activation energy for enterprise software sold through traditional channels is very high, so you'd have to be a *lot* better to get users to switch. whereas the activation energy required to switch to a new search engine is low. which in turn is why search engines are so much better than enterprise software.

4 this gets harder as you get older. while the space of ideas doesn't have dangerous local maxima, the space of careers does. there are fairly high walls between most of the paths people take through life, and the older you get, the higher the walls become.

5 it was also obvious to us that the web was going to be a big deal. few non-programmers grasped that in 1995, but the programmers had seen what guis had done for desktop computers.

6 maybe it would work to have this second self keep a journal, and each night to make a brief entry listing the gaps and anomalies you'd noticed that day. not startup ideas, just the raw gaps and anomalies.

7 sam altman points out that taking time to come up with an idea is not merely a better strategy in an absolute sense, but also like an undervalued stock in that so few founders do it.

there's comparatively little competition for the best ideas, because few founders are willing to put in the time required to notice them. whereas there is a great deal of competition for mediocre ideas, because when people make up startup ideas, they tend to make up the same ones.

8 for the computer hardware and software companies, summer jobs are the first phase of the recruiting funnel. but if you're good you can skip the first phase. if you're good you'll have no trouble getting hired by these companies when you graduate, regardless of how you spent your summers.

02

organic startup ideas

paulgraham.com/organic.html

hard not to end up making something of value to a lot of people, and when you do, surprise, you've got a company. [3]

don't be discouraged if what you produce initially is something other people dismiss as a toy. in fact, that's a good sign. that's probably why everyone else has been overlooking the idea. the first microcomputers were dismissed as toys. and the first planes, and the first cars. at this point, when someone comes to us with something that users like but that we could envision forum trolls dismissing as a toy, it makes us especially likely to invest.

while young founders are at a disadvantage when coming up with made-up ideas, they're the best source of organic ones, because they're at the forefront of technology. they use the latest stuff. they only just decided what to use, so why wouldn't they? and because they use the latest stuff, they're in a position to discover valuable types of fixable brokenness first.

there's nothing more valuable than an unmet need that is just becoming fixable. if you find something broken that you can fix for a lot of people, you've found a gold mine. as with an actual gold mine, you still have to work hard to get the gold out of it. but at least you know where the seam is, and that's the hard part.

---

notes

1 this suggests a way to predict areas where apple will be weak: things steve jobs doesn't use. e.g. i doubt he is much into gaming.

2 in retrospect, we should have *become* direct marketers. if i were doing viaweb again, i'd open our own online store. if we had, we'd have understood users a lot better. i'd encourage anyone starting a startup to become one of its users, however unnatural it seems.

3 possible exception: it's hard to compete directly with open source software. you can build things for programmers, but there has to be some part you can charge for.

thanks to sam altman, trevor blackwell, and jessica livingston for reading drafts of this.

03

schlep blindness

paulgraham.com/schlep.html

starting their company about the obstacles they'd have to overcome, they might never have started it. maybe that's one reason the most successful startups of all so often have young founders.

in practice the founders grow with the problems. but no one seems able to foresee that, not even older, more experienced founders. so the reason younger founders have an advantage is that they make two mistakes that cancel each other out. they don't know how much they can grow, but they also don't know how much they'll need to. older founders only make the first mistake.

ignorance can't solve everything though. some ideas so obviously entail alarming schleps that anyone can see them. how do you see ideas like that? the trick i recommend is to take yourself out of the picture. instead of asking "what problem should i solve?" ask "what problem do i wish someone else would solve for me?" if someone who had to process payments before stripe had tried asking that, stripe would have been one of the first things they wished for.

it's too late now to be stripe, but there's plenty still broken in the world, if you know how to see it.

thanks to sam altman, paul buchheit, patrick collison, aaron iba, jessica livingston, emmett shear, and harj taggar for reading drafts of this.

04

frighteningly ambitious startup ideas

paulgraham.com/ambitious.html

make it really good for code search, for example. would you like search queries to be turing complete? anything that gets you those 10,000 users is ipso facto good.

don't worry if something you want to do will constrain you in the long term, because if you don't get that initial core of users, there won't be a long term. if you can just build something that you and your friends genuinely prefer to google, you're already about 10% of the way to an ipo, just as facebook was (though they probably didn't realize it) when they got all the harvard undergrads.

2. replace email

email was not designed to be used the way we use it now. email is not a messaging protocol. it's a todo list. or rather, my inbox is a todo list, and email is the way things get onto it. but it is a disastrously bad todo list.

i'm open to different types of solutions to this problem, but i suspect that tweaking the inbox is not enough, and that email has to be replaced with a new protocol. this new protocol should be a todo list protocol, not a messaging protocol, although there is a degenerate case where what someone wants you to do is: read the following text.

as a todo list protocol, the new protocol should give more power to the recipient than email does. i want there to be more restrictions on what someone can put on my todo list. and when someone can put something on my todo list, i want them to tell me more about what they want from me. do they want me to do something beyond just reading some text? how important is it? (there obviously has to be some mechanism to prevent people from saying everything is important.) when does it have to be done?

this is one of those ideas that's like an irresistible force meeting an immovable object. on one hand, entrenched protocols are impossible to replace. on the other, it seems unlikely that people in 100 years will still be living in the same email hell we do now. and if email is going to get replaced eventually, why not now?

if you do it right, you may be able to avoid the usual chicken and egg problem new protocols face, because some of the most powerful people in the world will be among the first to switch to it. they're all at the mercy of email too.

whatever you build, make it fast. gmail has become painfully slow. [2] if you made something no better than gmail, but fast, that alone would let you start to pull users away from gmail.

gmail is slow because google can't afford to spend a lot on it. but people will pay for this. i'd have no problem paying $50 a month. considering how much time i spend in email, it's kind of scary to think how much i'd be justified in paying. at least $1000 a month. if i spend several hours a day reading and writing email, that would be a cheap way to make my life better.

3. replace universities

people are all over this idea lately, and i think they're onto something. i'm reluctant to suggest that an institution that's been around for a millennium is finished just because of some mistakes they made in the last few decades, but certainly in the last few decades us universities seem to have been headed down the wrong path. one could do a lot better for a lot less money.

i don't think universities will disappear. they won't be replaced wholesale. they'll just lose the de facto monopoly on certain types of learning that they once had. there will be many different ways to learn

what's possible. that helps would-be successors both directly, as roger bannister did, by showing how much better you can do than people did before, and indirectly, as augustus did, by lodging the idea in users' minds that a single person could unroll the future for them. [3]

now steve is gone there's a vacuum we can all feel. if a new company led boldly into the future of hardware, users would follow. the ceo of that company, the "next steve jobs," might not measure up to steve jobs. but he wouldn't have to. he'd just have to do a better job than samsung and hp and nokia, and that seems pretty doable.

6. bring back moore's law

the last 10 years have reminded us what moore's law actually says. till about 2002 you could safely misinterpret it as promising that clock speeds would double every 18 months. actually what it says is that circuit densities will double every 18 months. it used to seem pedantic to point that out. not any more. intel can no longer give us faster cpus, just more of them.

this moore's law is not as good as the old one. moore's law used to mean that if your software was slow, all you had to do was wait, and the inexorable progress of hardware would solve your problems. now if your software is slow you have to rewrite it to do more things in parallel, which is a lot more work than waiting.

it would be great if a startup could give us something of the old moore's law back, by writing software that could make a large number of cpus look to the developer like one very fast cpu. there are several ways to approach this problem. the most ambitious is to try to do it automatically: to write a compiler that will parallelize our code for us. there's a name for this compiler, *the sufficiently smart compiler,* and it is a byword for impossibility. but is it really impossible? is there no configuration of the bits in memory of a present day computer that is this compiler? if you really think so, you should try to prove it, because that would be an interesting result. and if it's not impossible but simply very hard, it might be worth trying to write it. the expected value would be high even if the chance of succeeding was low.

the reason the expected value is so high is web services. if you could write software that gave programmers the convenience of the way things were in the old days, you could offer it to them as a web service. and that would in turn mean that you got practically all the users.

imagine there was another processor manufacturer that could still translate increased circuit densities into increased clock speeds. they'd take most of intel's business. and since web services mean that no one sees their processors anymore, by writing the sufficiently smart compiler you could create a situation indistinguishable from you being that manufacturer, at least for the server market.

the least ambitious way of approaching the problem is to start from the other end, and offer programmers more parallelizable lego blocks to build programs out of, like hadoop and mapreduce. then the programmer still does much of the work of optimization.

there's an intriguing middle ground where you build a semi-automatic weapon—where there's a human in the loop. you make something that looks to the user like the sufficiently smart compiler, but inside has people, using highly developed optimization tools to find and eliminate bottlenecks in users' programs. these people might be your employees, or you might create a marketplace for optimization.

an optimization marketplace would be a way to generate the sufficiently smart compiler piecemeal, because participants would immediately start writing bots. it would be a curious state of affairs if you could get to the point where everything could be done by bots, because then you'd have made the sufficiently smart compiler, but no one person would have a complete copy of it.

let me conclude with some tactical advice. if you want to take on a problem as big as the ones i've discussed, don't make a direct frontal attack on it. don't say, for example, that you're going to replace email. if you do that you raise too many expectations. your employees and investors will constantly be asking "are we there yet?" and you'll have an army of haters waiting to see you fail. just say you're building todo-list software. that sounds harmless. people can notice you've replaced email when it's a *fait accompli*. [4]

empirically, the way to do really big things seems to be to start with deceptively small things. want to dominate microcomputer software? start by writing a basic interpreter for a machine with a few thousand users. want to make the universal web site? start by building a site for harvard undergrads to stalk one another.

empirically, it's not just for other people that you need to start small. you need to for your own sake. neither bill gates nor mark zuckerberg knew at first how big their companies were going to get. all they knew was that they were onto something. maybe it's a bad idea to have really big ambitions initially, because the bigger your ambition, the longer it's going to take, and the further you project into the future, the more likely you'll get it wrong.

i think the way to use these big ideas is not to try to identify a precise point in the future and then ask yourself how to get from here to there, like the popular image of a visionary. you'll be better off if you operate like columbus and just head in a general westerly direction. don't try to construct the future like a building, because your current blueprint is almost certainly mistaken. start with something you know works, and when you expand, expand westward.

the popular image of the visionary is someone with a clear view of the future, but empirically it may be better to have a blurry one.

---

notes

1 it's also one of the most important things vcs fail to understand about startups. most expect founders to walk in with a clear plan for the future, and judge them based on that. few consciously realize that in the biggest successes there is the least correlation between the initial plan and what the startup eventually becomes.

2 this sentence originally read "gmail is painfully slow." thanks to paul buchheit for the correction.

3 roger bannister is famous as the first person to run a mile in under 4 minutes. but his world record only lasted 46 days. once he showed it could be done, lots of others followed. ten years later jim ryun ran a 3:59 mile as a high school junior.

4 if you want to be the next apple, maybe you don't even want to start with consumer electronics. maybe at first you make something hackers use. or you make something popular but apparently unimportant, like a headset or router. all you need is a bridgehead.

thanks to sam altman, trevor blackwell, paul buchheit, patrick collison, aaron iba, jessica livingston, robert morris, harj taggar and garry tan for reading drafts of this.

05

do things that don't scale

paulgraham.com/ds.html

airbnb now seems like an unstoppable juggernaut, but early on it was so fragile that about 30 days of going out and engaging in person with users made the difference between success and failure.

that initial fragility was not a unique feature of airbnb. almost all startups are fragile initially. and that's one of the biggest things inexperienced founders and investors (and reporters and know-it-alls on forums) get wrong about them. they unconsciously judge larval startups by the standards of established ones. they're like someone looking at a newborn baby and concluding "there's no way this tiny creature could ever accomplish anything."

it's harmless if reporters and know-it-alls dismiss your startup. they always get things wrong. it's even ok if investors dismiss your startup; they'll change their minds when they see growth. the big danger is that you'll dismiss your startup yourself. i've seen it happen. i often have to encourage founders who don't see the full potential of what they're building. even bill gates made that mistake. he returned to harvard for the fall semester after starting microsoft. he didn't stay long, but he wouldn't have returned at all if he'd realized microsoft was going to be even a fraction of the size it turned out to be. [4]

the question to ask about an early stage startup is not "is this company taking over the world?" but "how big could this company get if the founders did the right things?" and the right things often seem both laborious and inconsequential at the time. microsoft can't have seemed very impressive when it was just a couple guys in albuquerque writing basic interpreters for a market of a few thousand hobbyists (as they were then called), but in retrospect that was the optimal path to dominating microcomputer software. and i know brian chesky and joe gebbia didn't feel like they were en route to the big time as they were taking "professional" photos of their first hosts' apartments. they were just trying to survive. but in retrospect that too was the optimal path to dominating a big market.

how do you find users to recruit manually? if you build something to solve your own problems, then you only have to find your peers, which is usually straightforward. otherwise you'll have to make a more deliberate effort to locate the most promising vein of users. the usual way to do that is to get some initial set of users by doing a comparatively untargeted launch, and then to observe which kind seem most enthusiastic, and seek out more like them. for example, ben silbermann noticed that a lot of the earliest pinterest users were interested in design, so he went to a conference of design bloggers to recruit users, and that worked well. [5]

delight

you should take extraordinary measures not just to acquire users, but also to make them happy. for as long as they could (which turned out to be surprisingly long), wufoo sent each new user a hand-written thank you note. your first users should feel that signing up with you was one of the best choices they ever made. and you in turn should be racking your brains to think of new ways to delight them.

why do we have to teach startups this? why is it counterintuitive for founders? three reasons, i think.

one is that a lot of startup founders are trained as engineers, and customer service is not part of the training of engineers. you're supposed to build things that are robust and elegant, not be slavishly attentive to individual users like some kind of salesperson. ironically, part of the reason engineering is traditionally averse to handholding is that its traditions date from a time when engineers were less powerful — when they were only in charge of their narrow domain of building things, rather than running the whole show. you can be ornery when you're scotty, but not when you're kirk.

another reason founders don't focus enough on individual customers is that they worry it won't scale. but when founders of larval startups worry about this, i point out that in their current state they have nothing to lose. maybe if they go out of their way to make existing users super happy, they'll one day

fire

sometimes the right unscalable trick is to focus on a deliberately narrow market. it's like keeping a fire contained at first to get it really hot before adding more logs.

that's what facebook did. at first it was just for harvard students. in that form it only had a potential market of a few thousand people, but because they felt it was really for them, a critical mass of them signed up. after facebook stopped being for harvard students, it remained for students at specific colleges for quite a while. when i interviewed mark zuckerberg at startup school, he said that while it was a lot of work creating course lists for each school, doing that made students feel the site was their natural home.

any startup that could be described as a marketplace usually has to start in a subset of the market, but this can work for other startups as well. it's always worth asking if there's a subset of the market in which you can get a critical mass of users quickly. [8]

most startups that use the contained fire strategy do it unconsciously. they build something for themselves and their friends, who happen to be the early adopters, and only realize later that they could offer it to a broader market. the strategy works just as well if you do it unconsciously. the biggest danger of not being consciously aware of this pattern is for those who naively discard part of it. e.g. if you don't build something for yourself and your friends, or even if you do, but you come from the corporate world and your friends are not early adopters, you'll no longer have a perfect initial market handed to you on a platter.

among companies, the best early adopters are usually other startups. they're more open to new things both by nature and because, having just been started, they haven't made all their choices yet. plus when they succeed they grow fast, and you with them. it was one of many unforeseen advantages of the yc model (and specifically of making yc big) that b2b startups now have an instant market of hundreds of other startups ready at hand.

meraki

for hardware startups there's a variant of doing things that don't scale that we call "pulling a meraki." although we didn't fund meraki, the founders were robert morris's grad students, so we know their history. they got started by doing something that really doesn't scale: assembling their routers themselves.

hardware startups face an obstacle that software startups don't. the minimum order for a factory production run is usually several hundred thousand dollars. which can put you in a catch-22: without a product you can't generate the growth you need to raise the money to manufacture your product. back when hardware startups had to rely on investors for money, you had to be pretty convincing to overcome this. the arrival of crowdfunding (or more precisely, preorders) has helped a lot. but even so i'd advise startups to pull a meraki initially if they can. that's what pebble did. the pebbles assembled the first several hundred watches themselves. if they hadn't gone through that phase, they probably wouldn't have sold $10 million worth of watches when they did go on kickstarter.

like paying excessive attention to early customers, fabricating things yourself turns out to be valuable for hardware startups. you can tweak the design faster when you're the factory, and you learn things you'd never have known otherwise. eric migicovsky of pebble said one of the things he learned was "how valuable it was to source good screws." who knew?

consult

few months later will depend more on how happy you made those users than how many there were of them. [10]

so why do founders think launches matter? a combination of solipsism and laziness. they think what they're building is so great that everyone who hears about it will immediately sign up. plus it would be so much less work if you could get users merely by broadcasting your existence, rather than recruiting them one at a time. but even if what you're building really is great, getting users will always be a gradual process — partly because great things are usually also novel, but mainly because users have other things to think about.

partnerships too usually don't work. they don't work for startups in general, but they especially don't work as a way to get growth started. it's a common mistake among inexperienced founders to believe that a partnership with a big company will be their big break. six months later they're all saying the same thing: that was way more work than we expected, and we ended up getting practically nothing out of it. [11]

it's not enough just to do something extraordinary initially. you have to make an extraordinary *effort* initially. any strategy that omits the effort — whether it's expecting a big launch to get you users, or a big partner — is ipso facto suspect.

vector

the need to do something unscalably laborious to get started is so nearly universal that it might be a good idea to stop thinking of startup ideas as scalars. instead we should try thinking of them as pairs of what you're going to build, plus the unscalable thing(s) you're going to do initially to get the company going.

it could be interesting to start viewing startup ideas this way, because now that there are two components you can try to be imaginative about the second as well as the first. but in most cases the second component will be what it usually is — recruit users manually and give them an overwhelmingly good experience — and the main benefit of treating startups as vectors will be to remind founders they need to work hard in two dimensions. [12]

in the best case, both components of the vector contribute to your company's dna: the unscalable things you have to do to get started are not merely a necessary evil, but change the company permanently for the better. if you have to be aggressive about user acquisition when you're small, you'll probably still be aggressive when you're big. if you have to manufacture your own hardware, or use your software on users's behalf, you'll learn things you couldn't have learned otherwise. and most importantly, if you have to work hard to delight users when you only have a handful of them, you'll keep doing it when you have a lot.

---

notes

1 actually emerson never mentioned mousetraps specifically. he wrote "if a man has good corn or wood, or boards, or pigs, to sell, or can make better chairs or knives, crucibles or church organs, than anybody else, you will find a broad hard-beaten road to his house, though it be in the woods."

2 thanks to sam altman for suggesting i make this explicit. and no, you can't avoid doing sales by hiring someone to do it for you. you have to do sales yourself initially. later you can hire a real salesperson to replace you.

06

startup = growth

paulgraham.com/growth.html

icient market. if you write software to teach tibetan to hungarians, you won't have much competition. if you write software to teach english to chinese speakers, you'll face ferocious competition, precisely because that's such a larger prize. [2]

the constraints that limit ordinary companies also protect them. that's the tradeoff. if you start a barbershop, you only have to compete with other local barbers. if you start a search engine you have to compete with the whole world.

the most important thing that the constraints on a normal business protect it from is not competition, however, but the difficulty of coming up with new ideas. if you open a bar in a particular neighborhood, as well as limiting your potential and protecting you from competitors, that geographic constraint also helps define your company. bar + neighborhood is a sufficient idea for a small business. similarly for companies constrained in (a). your niche both protects and defines you.

whereas if you want to start a startup, you're probably going to have to think of something fairly novel. a startup has to make something it can deliver to a large market, and ideas of that type are so valuable that all the obvious ones are already taken.

that space of ideas has been so thoroughly picked over that a startup generally has to work on something everyone else has overlooked. i was going to write that one has to make a conscious effort to find ideas everyone else has overlooked. but that's not how most startups get started. usually successful startups happen because the founders are sufficiently different from other people that ideas few others can see seem obvious to them. perhaps later they step back and notice they've found an idea in everyone else's blind spot, and from that point make a deliberate effort to stay there. [3] but at the moment when successful startups get started, much of the innovation is unconscious.

what's different about successful founders is that they can see different problems. it's a particularly good combination both to be good at technology and to face problems that can be solved by it, because technology changes so rapidly that formerly bad ideas often become good without anyone noticing. steve wozniak's problem was that he wanted his own computer. that was an unusual problem to have in 1975. but technological change was about to make it a much more common one. because he not only wanted a computer but knew how to build them, wozniak was able to make himself one. and the problem he solved for himself became one that apple solved for millions of people in the coming years. but by the time it was obvious to ordinary people that this was a big market, apple was already established.

google has similar origins. larry page and sergey brin wanted to search the web. but unlike most people they had the technical expertise both to notice that existing search engines were not as good as they could be, and to know how to improve them. over the next few years their problem became everyone's problem, as the web grew to a size where you didn't have to be a picky search expert to notice the old algorithms weren't good enough. but as happened with apple, by the time everyone else realized how important search was, google was entrenched.

that's one connection between startup ideas and technology. rapid change in one area uncovers big, soluble problems in other areas. sometimes the changes are advances, and what they change is solubility. that was the kind of change that yielded apple; advances in chip technology finally let steve wozniak design a computer he could afford. but in google's case the most important change was the growth of the web. what changed there was not solubility but bigness.

the other connection between startups and technology is that startups create new ways of doing things, and new ways of doing things are, in the broader sense of the word, new technology. when a startup

we usually advise startups to pick a growth rate they think they can hit, and then just try to hit it every week. the key word here is "just." if they decide to grow at 7% a week and they hit that number, they're successful for that week. there's nothing more they need to do. but if they don't hit it, they've failed in the only thing that mattered, and should be correspondingly alarmed.

programmers will recognize what we're doing here. we're turning starting a startup into an optimization problem. and anyone who has tried optimizing code knows how wonderfully effective that sort of narrow focus can be. optimizing code means taking an existing program and changing it to use less of something, usually time or memory. you don't have to think about what the program should do, just make it faster. for most programmers this is very satisfying work. the narrow focus makes it a sort of puzzle, and you're generally surprised how fast you can solve it.

focusing on hitting a growth rate reduces the otherwise bewilderingly multifarious problem of starting a startup to a single problem. you can use that target growth rate to make all your decisions for you; anything that gets you the growth you need is ipso facto right. should you spend two days at a conference? should you hire another programmer? should you focus more on marketing? should you spend time courting some big customer? should you add x feature? whatever gets you your target growth rate. [8]

judging yourself by weekly growth doesn't mean you can look no more than a week ahead. once you experience the pain of missing your target one week (it was the only thing that mattered, and you failed at it), you become interested in anything that could spare you such pain in the future. so you'll be willing for example to hire another programmer, who won't contribute to this week's growth but perhaps in a month will have implemented some new feature that will get you more users. but only if (a) the distraction of hiring someone won't make you miss your numbers in the short term, and (b) you're sufficiently worried about whether you can keep hitting your numbers without hiring someone new.

it's not that you don't think about the future, just that you think about it no more than necessary.

in theory this sort of hill-climbing could get a startup into trouble. they could end up on a local maximum. but in practice that never happens. having to hit a growth number every week forces founders to act, and acting versus not acting is the high bit of succeeding. nine times out of ten, sitting around strategizing is just a form of procrastination. whereas founders' intuitions about which hill to climb are usually better than they realize. plus the maxima in the space of startup ideas are not spiky and isolated. most fairly good ideas are adjacent to even better ones.

the fascinating thing about optimizing for growth is that it can actually discover startup ideas. you can use the need for growth as a form of evolutionary pressure. if you start out with some initial plan and modify it as necessary to keep hitting, say, 10% weekly growth, you may end up with a quite different company than you meant to start. but anything that grows consistently at 10% a week is almost certainly a better idea than you started with.

there's a parallel here to small businesses. just as the constraint of being located in a particular neighborhood helps define a bar, the constraint of growing at a certain rate can help define a startup.

you'll generally do best to follow that constraint wherever it leads rather than being influenced by some initial vision, just as a scientist is better off following the truth wherever it leads rather than being influenced by what he wishes were the case. when richard feynman said that the imagination of nature was greater than the imagination of man, he meant that if you just keep following the truth

the test of any investment is the ratio of return to risk. startups pass that test because although they're appallingly risky, the returns when they do succeed are so high. but that's not the only reason investors like startups. an ordinary slower-growing business might have just as good a ratio of return to risk, if both were lower. so why are vcs interested only in high-growth companies? the reason is that they get paid by getting their capital back, ideally after the startup ipos, or failing that when it's acquired.

the other way to get returns from an investment is in the form of dividends. why isn't there a parallel vc industry that invests in ordinary companies in return for a percentage of their profits? because it's too easy for people who control a private company to funnel its revenues to themselves (e.g. by buying overpriced components from a supplier they control) while making it look like the company is making little profit. anyone who invested in private companies in return for dividends would have to pay close attention to their books.

the reason vcs like to invest in startups is not simply the returns, but also because such investments are so easy to oversee. the founders can't enrich themselves without also enriching the investors. [12]

why do founders want to take the vcs' money? growth, again. the constraint between good ideas and growth operates in both directions. it's not merely that you need a scalable idea to grow. if you have such an idea and don't grow fast enough, competitors will. growing too slowly is particularly dangerous in a business with network effects, which the best startups usually have to some degree.

almost every company needs some amount of funding to get started. but startups often raise money even when they are or could be profitable. it might seem foolish to sell stock in a profitable company for less than you think it will later be worth, but it's no more foolish than buying insurance. fundamentally that's how the most successful startups view fundraising. they could grow the company on its own revenues, but the extra money and help supplied by vcs will let them grow even faster. raising money lets you *choose* your growth rate.

money to grow faster is always at the command of the most successful startups, because the vcs need them more than they need the vcs. a profitable startup could if it wanted just grow on its own revenues. growing slower might be slightly dangerous, but chances are it wouldn't kill them. whereas vcs need to invest in startups, and in particular the most successful startups, or they'll be out of business. which means that any sufficiently promising startup will be offered money on terms they'd be crazy to refuse. and yet because of the scale of the successes in the startup business, vcs can still make money from such investments. you'd have to be crazy to believe your company was going to become as valuable as a high growth rate can make it, but some do.

pretty much every successful startup will get acquisition offers too. why? what is it about startups that makes other companies want to buy them? [13]

fundamentally the same thing that makes everyone else want the stock of successful startups: a rapidly growing company is valuable. it's a good thing ebay bought paypal, for example, because paypal is now responsible for 43% of their sales and probably more of their growth.

but acquirers have an additional reason to want startups. a rapidly growing company is not merely valuable, but dangerous. if it keeps expanding, it might expand into the acquirer's own territory. most product acquisitions have some component of fear. even if an acquirer isn't threatened by the startup itself, they might be alarmed at the thought of what a competitor could do with it. and because startups are in this sense doubly valuable to acquirers, acquirers will often pay more than an ordinary investor would. [14]

understand

4 i got it wrong in "how to make wealth" when i said that a startup was a small company that takes on a hard technical problem. that is the most common recipe but not the only one.

5 in principle companies aren't limited by the size of the markets they serve, because they could just expand into new markets. but there seem to be limits on the ability of big companies to do that. which means the slowdown that comes from bumping up against the limits of one's markets is ultimately just another way in which internal limits are expressed.

it may be that some of these limits could be overcome by changing the shape of the organization — specifically by sharding it.

6 this is, obviously, only for startups that have already launched or can launch during yc. a startup building a new database will probably not do that. on the other hand, launching something small and then using growth rate as evolutionary pressure is such a valuable technique that any company that could start this way probably should.

7 if the startup is taking the facebook/twitter route and building something they hope will be very popular but from which they don't yet have a definite plan to make money, the growth rate has to be higher, even though it's a proxy for revenue growth, because such companies need huge numbers of users to succeed at all.

beware too of the edge case where something spreads rapidly but the churn is high as well, so that you have good net growth till you run through all the potential users, at which point it suddenly stops.

8 within yc when we say it's ipso facto right to do whatever gets you growth, it's implicit that this excludes trickery like buying users for more than their lifetime value, counting users as active when they're really not, bleeding out invites at a regularly increasing rate to manufacture a perfect growth curve, etc. even if you were able to fool investors with such tricks, you'd ultimately be hurting yourself, because you're throwing off your own compass.

9 which is why it's such a dangerous mistake to believe that successful startups are simply the embodiment of some brilliant initial idea. what you're looking for initially is not so much a great idea as an idea that could evolve into a great one. the danger is that promising ideas are not merely blurry versions of great ones. they're often different in kind, because the early adopters you evolve the idea upon have different needs from the rest of the market. for example, the idea that evolves into facebook isn't merely a subset of facebook; the idea that evolves into facebook is a site for harvard undergrads.

10 what if a company grew at 1.7x a year for a really long time? could it not grow just as big as any successful startup? in principle yes, of course. if our hypothetical company making $1000 a month grew at 1% a week for 19 years, it would grow as big as a company growing at 5% a week for 4 years. but while such trajectories may be common in, say, real estate development, you don't see them much in the technology business. in technology, companies that grow slowly tend not to grow as big.

11 any expected value calculation varies from person to person depending on their utility function for money. i.e. the first million is worth more to most people than subsequent millions. how much more depends on the person. for founders who are younger or more ambitious the utility function is flatter. which is probably part of the reason the founders of the most successful startups of all tend to be on the young side.

12 more precisely, this is the case in the biggest winners, which is where all the returns come from. a startup founder could pull the same trick of enriching himself at the company's expense by selling them overpriced components. but it wouldn't be worth it for the founders of google to do that. only founders of failing startups would even be tempted, but those are writeoffs from the vcs' point of view anyway.

13 acquisitions fall into two categories: those where the acquirer wants the business, and those where the acquirer just wants the employees. the latter type is sometimes called an hr acquisition. though nominally acquisitions and sometimes on a scale that has a significant effect on the expected value calculation for potential founders, hr acquisitions are viewed by acquirers as more akin to hiring bonuses.

14 i once explained this to some founders who had recently arrived from russia. they found it novel that if you threatened a company they'd pay a premium for you. "in russia they just kill you," they said, and they were only partly joking. economically, the fact that established companies can't simply eliminate new competitors

07

before the startup

paulgraham.com/before.html

the second counterintuitive point is that it's not that important to know a lot about startups. the way to succeed in a startup is not to be an expert on startups, but to be an expert on your users and the problem you're solving for them. mark zuckerberg didn't succeed because he was an expert on startups. he succeeded despite being a complete noob at startups, because he understood his users really well.

if you don't know anything about, say, how to raise an angel round, don't feel bad on that account. that sort of thing you can learn when you need to, and forget after you've done it.

in fact, i worry it's not merely unnecessary to learn in great detail about the mechanics of startups, but possibly somewhat dangerous. if i met an undergrad who knew all about convertible notes and employee agreements and (god forbid) class ff stock, i wouldn't think "here is someone who is way ahead of their peers." it would set off alarms. because another of the characteristic mistakes of young founders is to go through the motions of starting a startup. they make up some plausible-sounding idea, raise money at a good valuation, rent a cool office, hire a bunch of people. from the outside that seems like what startups do. but the next step after rent a cool office and hire a bunch of people is: gradually realize how completely fucked they are, because while imitating all the outward forms of a startup they have neglected the one thing that's actually essential: making something people want.

game

we saw this happen so often that we made up a name for it: playing house. eventually i realized why it was happening. the reason young founders go through the motions of starting a startup is because that's what they've been trained to do for their whole lives up to that point. think about what you have to do to get into college, for example. extracurricular activities, check. even in college classes most of the work is as artificial as running laps.

i'm not attacking the educational system for being this way. there will always be a certain amount of fakeness in the work you do when you're being taught something, and if you measure their performance it's inevitable that people will exploit the difference to the point where much of what you're measuring is artifacts of the fakeness.

i confess i did it myself in college. i found that in a lot of classes there might only be 20 or 30 ideas that were the right shape to make good exam questions. the way i studied for exams in these classes was not (except incidentally) to master the material taught in the class, but to make a list of potential exam questions and work out the answers in advance. when i walked into the final, the main thing i'd be feeling was curiosity about which of my questions would turn up on the exam. it was like a game.

it's not surprising that after being trained for their whole lives to play such games, young founders' first impulse on starting a startup is to try to figure out the tricks for winning at this new game. since fundraising appears to be the measure of success for startups (another classic noob mistake), they always want to know what the tricks are for convincing investors. we tell them the best way to convince investors is to make a startup that's actually doing well, meaning growing fast, and then simply tell investors so. then they want to know what the tricks are for growing fast. and we have to tell them the best way to do that is simply to make something people want.

so many of the conversations yc partners have with young founders begin with the founder asking "how do we..." and the partner replying "just..."

why do the founders always make things so complicated? the reason, i realized, is that they're looking for the trick.

starting a successful startup is similar to having kids in that it's like a button you push that changes your life irrevocably. and while it's truly wonderful having kids, there are a lot of things that are easier to do before you have them than after. many of which will make you a better parent when you do have kids. and since you can delay pushing the button for a while, most people in rich countries do.

yet when it comes to startups, a lot of people seem to think they're supposed to start them while they're still in college. are you crazy? and what are the universities thinking? they go out of their way to ensure their students are well supplied with contraceptives, and yet they're setting up entrepreneurship programs and startup incubators left and right.

to be fair, the universities have their hand forced here. a lot of incoming students are interested in startups. universities are, at least de facto, expected to prepare them for their careers. so students who want to start startups hope universities can teach them about startups. and whether universities can do this or not, there's some pressure to claim they can, lest they lose applicants to other universities that do.

can universities teach students about startups? yes and no. they can teach students about startups, but as i explained before, this is not what you need to know. what you need to learn about are the needs of your own users, and you can't do that until you actually start the company. [5] so starting a startup is intrinsically something you can only really learn by doing it. and it's impossible to do that in college, for the reason i just explained: startups take over your life. you can't start a startup for real as a student, because if you start a startup for real you're not a student anymore. you may be nominally a student for a bit, but you won't even be that for long. [6]

given this dichotomy, which of the two paths should you take? be a real student and not start a startup, or start a real startup and not be a student? i can answer that one for you. do not start a startup in college. how to start a startup is just a subset of a bigger problem you're trying to solve: how to have a good life. and though starting a startup can be part of a good life for a lot of ambitious people, age 20 is not the optimal time to do it. starting a startup is like a brutally fast depth-first search. most people should still be searching breadth-first at 20.

you can do things in your early 20s that you can't do as well before or after, like plunge deeply into projects on a whim and travel super cheaply with no sense of a deadline. for unambitious people, this sort of thing is the dreaded "failure to launch," but for the ambitious ones it can be an incomparably valuable sort of exploration. if you start a startup at 20 and you're sufficiently successful, you'll never get to do it. [7]

mark zuckerberg will never get to bum around a foreign country. he can do other things most people can't, like charter jets to fly him to foreign countries. but success has taken a lot of the serendipity out of his life. facebook is running him as much as he's running facebook. and while it can be very cool to be in the grip of a project you consider your life's work, there are advantages to serendipity too, especially early in life. among other things it gives you more options to choose your life's work from.

there's not even a tradeoff here. you're not sacrificing anything if you forgo starting a startup at 20, because you're more likely to succeed if you wait. in the unlikely case that you're 20 and one of your side projects takes off like facebook did, you'll face a choice of running with it or not, and it may be reasonable to run with it. but the usual way startups take off is for the founders to make them take off, and it's gratuitously stupid to do that at 20.

try

the first time i wrote that paragraph, instead of "learn a lot about things that matter," i wrote "become good at some technology." but that prescription, though sufficient, is too narrow. what was special about brian chesky and joe gebbia was not that they were experts in technology. they were good at design, and perhaps even more importantly, they were good at organizing groups and making projects happen. so you don't have to work on technology per se, so long as you work on problems demanding enough to stretch you.

what kind of problems are those? that is very hard to answer in the general case. history is full of examples of young people who were working on important problems that no one else at the time thought were important, and in particular that their parents didn't think were important. on the other hand, history is even fuller of examples of parents who thought their kids were wasting their time and who were right. so how do you know when you're working on real stuff? [8]

i know how *i* know. real problems are interesting, and i am self-indulgent in the sense that i always want to work on interesting things, even if no one else cares about them (in fact, especially if no one else cares about them), and find it very hard to make myself work on boring things, even if they're supposed to be important.

my life is full of case after case where i worked on something just because it seemed interesting, and it turned out later to be useful in some worldly way. y combinator itself was something i only did because it seemed interesting. so i seem to have some sort of internal compass that helps me out. but i don't know what other people have in their heads. maybe if i think more about this i can come up with heuristics for recognizing genuinely interesting problems, but for the moment the best i can offer is the hopelessly question-begging advice that if you have a taste for genuinely interesting problems, indulging it energetically is the best way to prepare yourself for a startup. and indeed, probably also the best way to live. [9]

but although i can't explain in the general case what counts as an interesting problem, i can tell you about a large subset of them. if you think of technology as something that's spreading like a sort of fractal stain, every moving point on the edge represents an interesting problem. so one guaranteed way to turn your mind into the type that has good startup ideas is to get yourself to the leading edge of some technology — to cause yourself, as paul buchheit put it, to "live in the future." when you reach that point, ideas that will seem to other people uncannily prescient will seem obvious to you. you may not realize they're startup ideas, but you'll know they're something that ought to exist.

for example, back at harvard in the mid 90s a fellow grad student of my friends robert and trevor wrote his own voice over ip software. he didn't mean it to be a startup, and he never tried to turn it into one. he just wanted to talk to his girlfriend in taiwan without paying for long distance calls, and since he was an expert on networks it seemed obvious to him that the way to do it was turn the sound into packets and ship it over the internet. he never did any more with his software than talk to his girlfriend, but this is exactly the way the best startups get started.

so strangely enough the optimal thing to do in college if you want to be a successful startup founder is not some sort of new, vocational version of college focused on "entrepreneurship." it's the classic version of college as education for its own sake. if you want to start a startup after college, what you should do in college is learn powerful things. and if you have genuine intellectual curiosity, that's what you'll naturally tend to do if you just follow your own inclinations. [10]

the component of entrepreneurship that really matters is domain expertise. the way to become larry page was to become an expert on search. and the way to become an expert on search was to be driven

08

startups in 13 sentences

paulgraham.com/13sentences.html

ciously lowered by such experiences. try making your customer service not merely good, but surprisingly good. go out of your way to make people happy. they'll be overwhelmed; you'll see. in the earliest stages of a startup, it pays to offer customer service on a level that wouldn't scale, because it's a way of learning about your users.

7. you make what you measure.

i learned this one from joe kraus. [3] merely measuring something has an uncanny tendency to improve it. if you want to make your user numbers go up, put a big piece of paper on your wall and every day plot the number of users. you'll be delighted when it goes up and disappointed when it goes down. pretty soon you'll start noticing what makes the number go up, and you'll start to do more of that. corollary: be careful what you measure.

8. spend little.

i can't emphasize enough how important it is for a startup to be cheap. most startups fail before they make something people want, and the most common form of failure is running out of money. so being cheap is (almost) interchangeable with iterating rapidly. [4] but it's more than that. a culture of cheapness keeps companies young in something like the way exercise keeps people young.

9. get ramen profitable.

"ramen profitable" means a startup makes just enough to pay the founders' living expenses. it's not rapid prototyping for business models (though it can be), but more a way of hacking the investment process. once you cross over into ramen profitable, it completely changes your relationship with investors. it's also great for morale.

10. avoid distractions.

nothing kills startups like distractions. the worst type are those that pay money: day jobs, consulting, profitable side-projects. the startup may have more long-term potential, but you'll always interrupt working on it to answer calls from people paying you now. paradoxically, fundraising is this type of distraction, so try to minimize that too.

11. don't get demoralized.

though the immediate cause of death in a startup tends to be running out of money, the underlying cause is usually lack of focus. either the company is run by stupid people (which can't be fixed with advice) or the people are smart but got demoralized. starting a startup is a huge moral weight. understand this and make a conscious effort not to be ground down by it, just as you'd be careful to bend at the knees when picking up a heavy box.

12. don't give up.

even if you get demoralized, don't give up. you can get surprisingly far by just not giving up. this isn't true in all fields. there are a lot of people who couldn't become good mathematicians no matter how long they persisted. but startups aren't like that. sheer effort is usually enough, so long as you keep morphing your idea.

13. deals fall through.

one of the most useful skills we learned from viaweb was not getting our hopes up. we probably had 20 deals of various types fall through. after the first 10 or so we learned to treat deals as background

09

the reddits

march 2024
paulgraham.com/reddits.html

frontpage were chosen by human moderators. and though they did a good job, that one small difference turned out to be a big difference. being driven by user submissions meant reddit was fresher than slashdot. news there was newer, and users will always go where the newest news is.

i pushed the reddits to launch fast. a version one didn't need to be more than a couple hundred lines of code. how could that take more than a week or two to build? and they did launch comparatively fast, about three weeks into the first yc batch. the first users were steve, alexis, me, and some of their yc batchmates and college friends. it turns out you don't need that many users to collect a decent list of interesting links, especially if you have multiple accounts per user.

reddit got two more people from their yc batch: chris slowe and aaron swartz, and they too were unusually smart. chris was just finishing his phd in physics at harvard. aaron was younger, a college freshman, and even more anti-authority than steve. it's not exaggerating to describe him as a martyr for what authority later did to him.

slowly but inexorably reddit's traffic grew. at first the numbers were so small they were hard to distinguish from background noise. but within a few weeks it was clear that there was a core of real users returning regularly to the site. and although all kinds of things have happened to reddit the company in the years since, reddit the *site* never looked back.

reddit the site (and now app) is such a fundamentally useful thing that it's almost unkillable. which is why, despite a long stretch after steve left when the management strategy ranged from benign neglect to spectacular blunders, traffic just kept growing. you can't do that with most companies. most companies you take your eye off the ball for six months and you're in deep trouble. but reddit was special, and when steve came back in 2015, i knew the world was in for a surprise.

people thought they had reddit's number: one of the players in silicon valley, but not one of the big ones. but those who knew what had been going on behind the scenes knew there was more to the story than this. if reddit could grow to the size it had with management that was harmless at best, what could it do if steve came back? we now know the answer to that question. or at least a lower bound on the answer. steve is not out of ideas yet.

10

how to start google

march 2024
paulgraham.com/google.html

ies all make you take a coding test before they'll hire you, regardless of where you went to university or how well you did there. they know grades and exam results prove nothing.

if you really want to learn to program, you have to work on your own projects. you learn so much faster that way. imagine you're writing a game and there's something you want to do in it, and you don't know how. you're going to figure out how a lot faster than you'd learn anything in a class.

you don't have to learn programming, though. if you're wondering what counts as technology, it includes practically everything you could describe using the words "make" or "build." so welding would count, or making clothes, or making videos. whatever you're most interested in. the critical distinction is whether you're producing or just consuming. are you writing computer games, or just playing them? that's the cutoff.

steve jobs, the founder of apple, spent time when he was a teenager studying calligraphy — the sort of beautiful writing that you see in medieval manuscripts. no one, including him, thought that this would help him in his career. he was just doing it because he was interested in it. but it turned out to help him a lot. the computer that made apple really big, the macintosh, came out at just the moment when computers got powerful enough to make letters like the ones in printed books instead of the computery-looking letters you see in 8 bit games. apple destroyed everyone else at this, and one reason was that steve was one of the few people in the computer business who really got graphic design.

don't feel like your projects have to be *serious*. they can be as frivolous as you like, so long as you're building things you're excited about. probably 90% of programmers start out building games. they and their friends like to play games. so they build the kind of things they and their friends want. and that's exactly what you should be doing at 15 if you want to start a startup one day.

you don't have to do just one project. in fact it's good to learn about multiple things. steve jobs didn't just learn calligraphy. he also learned about electronics, which was even more valuable. whatever you're interested in. (do you notice a theme here?)

so that's the first of the three things you need, to get good at some kind or kinds of technology. you do it the same way you get good at the violin or football: practice. if you start a startup at 22, and you start writing your own programs now, then by the time you start the company you'll have spent at least 7 years practicing writing code, and you can get pretty good at anything after practicing it for 7 years.

let's suppose you're 22 and you've succeeded: you're now really good at some technology. how do you get startup ideas? it might seem like that's the hard part. even if you are a good programmer, how do you get the idea to start google?

actually it's easy to get startup ideas once you're good at technology. once you're good at some technology, when you look at the world you see dotted outlines around the things that are missing. you start to be able to see both the things that are missing from the technology itself, and all the broken things that could be fixed using it, and each one of these is a potential startup.

in the town near our house there's a shop with a sign warning that the door is hard to close. the sign has been there for several years. to the people in the shop it must seem like this mysterious natural phenomenon that the door sticks, and all they can do is put up a sign warning customers about it. but any carpenter looking at this situation would think "why don't you just plane off the part that sticks?"

once you're good at programming, all the missing software in the world starts to become as obvious as a sticking door to a carpenter. i'll give you a real world example. back in the 20th century, american universities used to publish printed directories with all the students' names and contact info. when i

what you need in a cofounder is someone who's good at what they do and that you work well with, and the only way to judge this is to work with them on things.

at this point i'm going to tell you something you might not want to hear. it really matters to do well in your classes, even the ones that are just memorization or blathering about literature, because you need to do well in your classes to get into a good university. and if you want to start a startup you should try to get into the best university you can, because that's where the best cofounders are. it's also where the best employees are. when larry and sergey started google, they began by just hiring all the smartest people they knew out of stanford, and this was a real advantage for them.

the empirical evidence is clear on this. if you look at where the largest numbers of successful startups come from, it's pretty much the same as the list of the most selective universities.

i don't think it's the prestigious names of these universities that cause more good startups to come out of them. nor do i think it's because the quality of the teaching is better. what's driving this is simply the difficulty of getting in. you have to be pretty smart and determined to get into mit or cambridge, so if you do manage to get in, you'll find the other students include a lot of smart and determined people. [3]

you don't have to start a startup with someone you meet at university. the founders of twitch met when they were seven. the founders of stripe, patrick and john collison, met when john was born. but universities are the main source of cofounders. and because they're where the cofounders are, they're also where the ideas are, because the best ideas grow out of projects you do with the people who become your cofounders.

so the list of what you need to do to get from here to starting a startup is quite short. you need to get good at technology, and the way to do that is to work on your own projects. and you need to do as well in school as you can, so you can get into a good university, because that's where the cofounders and the ideas are.

that's it, just two things, build stuff and do well in school.

---

notes

1 the rhetorical trick in this sentence is that the "google"s refer to different things. what i mean is: a company that has as much chance of growing as big as google ultimately did as larry and sergey could have reasonably expected google itself would at the time they started it. but i think the original version is zippier.

2 making something for your friends isn't the only source of startup ideas. it's just the best source for young founders, who have the least knowledge of what other people want, and whose own wants are most predictive of future demand.

3 strangely enough this is particularly true in countries like the us where undergraduate admissions are done badly. us admissions departments make applicants jump through a lot of arbitrary hoops that have little to do with their intellectual ability. but the more arbitrary a test, the more it becomes a test of mere determination and resourcefulness. and those are the two most important qualities in startup founders. so us admissions departments are better at selecting founders than they would be if they were better at selecting students.

thanks to jared friedman, carolynn levy, jessica livingston, harj taggar, and garry tan for reading drafts of this.

11

founder mode

september 2024
paulgraham.com/foundermode.html

the norm instead of a practice so unusual that there's a name for it. and once you abandon that constraint there are a huge number of permutations to choose from.

for example, steve jobs used to run an annual retreat for what he considered the 100 most important people at apple, and these were not the 100 people highest on the org chart. can you imagine the force of will it would take to do this at the average company? and yet imagine how useful such a thing could be. it could make a big company feel like a startup. steve presumably wouldn't have kept having these retreats if they didn't work. but i've never heard of another company doing this. so is it a good idea, or a bad one? we still don't know. that's how little we know about founder mode. [2]

obviously founders can't keep running a 2000 person company the way they ran it when it had 20. there's going to have to be some amount of delegation. where the borders of autonomy end up, and how sharp they are, will probably vary from company to company. they'll even vary from time to time within the same company, as managers earn trust. so founder mode will be more complicated than manager mode. but it will also work better. we already know that from the examples of individual founders groping their way toward it.

indeed, another prediction i'll make about founder mode is that once we figure out what it is, we'll find that a number of individual founders were already most of the way there — except that in doing what they did they were regarded by many as eccentric or worse. [3]

curiously enough it's an encouraging thought that we still know so little about founder mode. look at what founders have achieved already, and yet they've achieved this against a headwind of bad advice. imagine what they'll do once we can tell them how to run their companies like steve jobs instead of john sculley.

---

notes

1 the more diplomatic way of phrasing this statement would be to say that experienced c-level execs are often very skilled at managing up. and i don't think anyone with knowledge of this world would dispute that.

2 if the practice of having such retreats became so widespread that even mature companies dominated by politics started to do it, we could quantify the senescence of companies by the average depth on the org chart of those invited.

3 i also have another less optimistic prediction: as soon as the concept of founder mode becomes established, people will start misusing it. founders who are unable to delegate even things they should will use founder mode as the excuse. or managers who aren't founders will decide they should try to act like founders. that may even work, to some extent, but the results will be messy when it doesn't; the modular approach does at least limit the damage a bad ceo can do.

thanks to brian chesky, patrick collison, ron conway, jessica livingston, elon musk, ryan petersen, harj taggar, and garry tan for reading drafts of this.

12

how universities should prepare founders

august 2026
paulgraham.com/prepare.html

they seem impressive, but they also seem human. especially when they talk about the early years, when they were clueless and made lots of mistakes. so strangely enough seeing founders in real life makes being one seem simultaneously both desirable and accessible. it makes students think "i want to be like that, and i could."

how inspiring founders are to students is a function roughly of how rich and famous they are divided by how much older they are than the students. so it's not essential to bring famous billionaires to campus. founders in their mid twenties who are 3 years into a startup with a valuation of a couple hundred million will do as well; they may only be a twentieth as rich and famous, but they're twenty times easier for students to identify with.

---

it's obvious why universities that want their students to start startups need to make them believe it's a viable option. but why is it so important for students to work on their own projects?

there are four reasons. the first is simply that it's a great way, possibly the best way, to understand a subject really deeply. the excitement of creating something new is a much more powerful motivator than the fear of doing badly on an exam.

second, working on projects together is the best way for cofounders to discover one another. the most successful startups tend to have multiple founders, and the only way to tell for sure if someone will be good to work with is to work with them. apple and microsoft were just the last of many projects their founders had worked on together.

third, a startup is a project, so starting one will feel natural to someone who's used to working on projects of their own. it won't seem weird that there's no teacher or boss telling them what to do. they're used to telling themselves.

fourth, and perhaps most surprisingly, random side projects are where the best startup ideas come from. the best startup ideas tend to seem so implausible at first that anyone consciously looking for startup ideas would reject them. who'd expect to start a huge company by creating a student directory? so the way to discover the best startup ideas is not to look for startup ideas but just to work on whatever random projects seem interesting. because in fact such projects are far from random: young people who are good at building things are technological bellwethers, so any idea that seems interesting to them is disproportionately likely to lead somewhere valuable, even if they themselves don't realize it yet.

now it should be clear why the yc partners care a lot about the projects that applicants have worked on and not at all about their gpas. projects are the best source of knowledge, the best source of founding teams, and the best source of startup ideas.

but encouraging students to work on their own projects may be difficult for universities. it will mean giving the students more free time, and universities may not like to do that.

microsoft and meta have something in common that few people realize. they both got started during reading period at harvard. reading period is the gap between the end of classes and the beginning of final exams. it's called reading period because students are *supposed* to spend it preparing for exams. but reading period also turns out to have the unique combination of qualities that make it perfect for starting new projects: the students are all on campus, and they don't have anything due the next day. that latter constraint, especially, is a huge drag on the most ambitious students. merely eliminating it for a

one common response to this inconvenient truth is to pretend to teach them how to start startups, for example by organizing business plan competitions. the students collaborate to come up with a startup idea, which they then pitch to simulated investors. this kind of exercise is not merely useless but positively misleading. it trains founders to think that fundraising is the essential step in starting a startup — that the core of starting a startup is to create a story that appeals to investors. as an investor, i can tell you that's not true. fundraising is merely a necessary evil. the people you need to impress are users, not investors, and the way you impress them is with prototypes, not words. the core of starting a startup is not creating a story that appeals to investors, but creating a product that appeals to users. [5]

in fact would-be founders should be doing exactly the opposite of what students do in business plan competitions. instead of thinking about startups without building anything, they should be building things without thinking about whether they'll turn into startups.

probably one of the reasons universities are tempted to organize bogus things like business plan competitions is that if they actually took the optimal measures to prepare students to start startups, it would look too quiet. imagine if a university were doing everything right. students would be getting a deep knowledge of how to build things in classes they were taking out of genuine interest, and working eagerly with their friends on side projects that had nothing to do with school. the students would graduate with exactly what predicts success in founders: the ability to build things and a habit of doing it. plus a significant number of those side projects would be incipient startups. and yet it would look to parents and prospective students as if the university wasn't doing anything. where are the classes on "entrepreneurship"? where is the innovation center?

and indeed this is another great thing about the optimal plan for preparing startup founders: it costs nothing extra. you don't have to hire any deans of entrepreneurship or build any new buildings. in fact if you do, those things will tend to drag you down; there's no need for them, so if they have any effect at all it will tend to be for the worse. if you have spare money, give it to the people teaching computer science or mechanical engineering or molecular biology. [6]

but if the optimal route looks too quiet, the solution is not to avoid it. the solution is to stand firm, knowing that you're doing the right thing, and eventually the results will speak for themselves. if you can develop an organic startup culture among your students and there are multiple students in every year who go on to start successful ones, this will soon become evident to anyone paying attention.

---

notes

1 should students still study computer science if ais will write most code? definitely. cs is an interesting subject in its own right and also a great way to understand problem solving in general. and even if you have ais writing all your code for you, you're still in the position of an engineering manager, and good engineering managers should be able to do the work of those working for them.

2 one reason i always put "entrepreneurship" in quotes is that it's a misleading word to use to describe starting startups. "entrepreneurship" simply means starting one's own business, and startups are a microscopically small subset of that world in which the rules are completely different. so conflating the two is asking for trouble.

3 of course all departments will claim to be teaching powerful ideas. but false claims of this type don't seem much of a danger. the sort of people who'd make good founders wouldn't even need to see through them; they simply wouldn't be interested enough in the classes taught by such departments to have much of their time wasted by them.

13

how to earn a billion dollars

june 2026
paulgraham.com/earn.html

that is how many months of 93% growth it takes to become a billionaire, starting from 2 million. a couple million and 93% growth are not, in fact, radically different from a billion. they're nine and a half months apart.

now you see why, when i meet a founder, the first thing i ask about is their growth rate.

but i don't want anyone to accuse me of using unrealistic numbers, so let's take a more conservative growth rate. let's see what happens at 15% a month. that's not rare at all. i constantly encounter startups growing at 15% a month.

if your revenues grow at 15% a month, how much more will you be making 5 years from now? to calculate that, we need to find 1.15 to the 60th power (since 5 years is 60 months). so go to google again and this time type 1.15^60. the answer should be about 4384. meaning in 5 years your startup will be making 4384 times as much. if you're currently making ten thousand a month, in five years you'll be making about 44 million a month, or 526 million a year. and at that point, if you own as much of the company as founders typically do, you will be a billionaire.

in the real world, growth rates tend to slow down a bit. a very successful startup will probably be growing faster than 15% a month in year 1 and slower than 15% a month in year 5. but you end up in about the same place. if you start a startup in your early twenties, it's definitely possible to be a billionaire by the time you're thirty. hard, but possible.

i wanted you to feel this by doing the calculation yourselves, because now you understand one of the reasons people start startups. exponential growth is like magic. it generates outcomes that seem impossible. and that's why some politicians distrust it. they don't understand the math of exponential growth, so when they see people becoming what seems to them impossibly rich, they assume they must have cheated.

but now you at least understand, from having done the math yourselves, that you don't have to cheat to become a billionaire. you've seen for yourselves that there are only two numbers in the calculation, the growth rate and how long it continues. if it's impossible to make a billion dollars without cheating, which of those two numbers is impossible? it's certainly not impossible to grow at 15% a month without cheating. startups do that all the time. and how long you can continue to grow at that rate depends on the size of the market. obviously for you to grow 4000x, there has to be at least 4000x more demand. but that's all you need. and how could you possibly cheat to increase the market size?

if you're only planning to become prime minister, you can stop paying attention now. we've proved that it is in fact possible to earn a billion dollars, because it only depends on two numbers, one of which startups routinely hit without cheating, and another that cheating couldn't possibly affect.

but if you actually want to become a billionaire, we should go into more detail. especially about that first number, the growth rate. to grow at a consistent rate every month, you have to make something so good that people tell their friends about it. and in fact that's the other reason i always begin by asking founders their growth rate. it shows whether they've built the right thing.

so how, exactly, do you make something people like so much that they tell their friends about it? the problem with market economies, and also the great thing about market economies, is that it's hard to make something customers want that they don't already have. as soon as a new, satisfiable need is discovered, people rush to satisfy it. so you're going to have to discover a need that no one else knows about yet.

how do you do that? *by feeling the need yourself.*

tanding users very deeply can you make something they love so much that they tell their friends about it, and only that can get you the exponential growth you need to make a startup really successful.

there are other ways to get rich than by starting startups. some of those do require you to exploit people. but startups are the most common way to become really rich, and if you want to start a successful startup, the key is not exploitation but empathy. what do users really want? what could you do for them that would make their lives dramatically better? that kind of empathy is what we look for in founders, and what we cultivate in the ones we accept.

how people become rich in your society is one of the most important things to understand about it. you can't let your beliefs about this be determined by ideology, or movies, or historical examples that are centuries old. you must look at the world around you and see how it's actually done. if you want to do it yourself, obviously you'll be forced to understand how it's done. so i don't worry too much about you. the ones i worry about are the future prime ministers. you need to remember this talk. so for you i'm going to summarize the key ideas.

there are two numbers that determine how big a startup gets, and thus how rich its founders become: the growth rate and how long it continues. you get the first by making something users like so much they tell their friends. you get the second by being in a big market. if you grow exponentially into a big market, your startup will become valuable, and you, as a shareholder, will become rich. you not only don't have to cheat to make this happen, it will happen automatically if you just keep making customers happy.

thanks to trevor blackwell, jared friedman, jessica livingston, and garry tan for reading drafts of this, and to arwa elrayess and the oxford union for hosting me.